

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по индивидуальному домашнему заданию
по дисциплине «Распределенные алгоритмы»

Тема: Реализация алгоритма на языке Promela и верификация в Spin

Студенты гр. 3343

Бондаренко Ф.А.

Пименов П.В.

Преподаватель

Шошмина И.В.

Санкт-Петербург

2026

Задание.

- Разработайте параллельный алгоритм умножения двух матриц $n \times n$ для P процессов так, чтобы параллельно вычислялись строки результирующей матрицы. Исходные матрицы загружены на процессы заранее.
- Сформулировать свойства алгоритма на языке LTL.
- Обмен данными, если он необходим, осуществляется с помощью каналов.
- По завершении умножения 0-й процесс выводит результат.

Выполнение работы.

1. Структура данных матриц

Алгоритм работает с тремя квадратными матрицами размера $N \times N$: двумя исходными A и B и результирующей C , где $C[i][j] = \sum A[i][k] \cdot B[k][j]$. По заданию исходные матрицы загружаются на процессы заранее.

В виду ограниченности Promela — каждая матрица представлена как плоский одномерный массив длины $N \times N$. Соответствие между логической парой индексов (i, j) и физическим индексом в одномерном массиве задается вручную по формуле $\text{индекс в массиве} = i \cdot N + j$.

Исходные матрицы A и B заполняются константами в блоке *init* до запуска рабочих процессов.

2. Распределение работы между процессами

Используется циклическое разбиение строк: создается P рабочих процессов с идентификаторами $id \in 0, 1, \dots, P-1$. Процесс с идентификатором id отвечает за строки результирующей матрицы с индексами $id, id+P, id+2P, \dots$.

«Строки» при этом не отдельные объекты — они определяются исключительно структурой обращения по индексам. В коде воркера ведется локальная переменная *row*, инициализированная значением id и получающая шаг P после каждой обработанной строки.

Никакой проверки принадлежности строки процессу не требуется: условие $row < N$ и инкремент $row = row + P$ гарантируют, что каждый воркер обходит только свои строки, таким образом, строки разных воркеров не пересекаются. Тем самым, исключая гонки записи в результирующую матрицу C .

Запуск процессов выполняется в блоке *init* в атомарном цикле, чтобы все экземпляры стартовали детерминированно одним шагом.

3. Алгоритм умножения (вычисления одной ячейки)

Для каждой пары (row, col) , относящейся к данному процессу, вычисляется скалярное произведение строки row матрицы A и столбца col матрицы B :
$$C[row][col] = \sum A[row][k] \cdot B[k][col].$$

Тройной цикл выражен тремя вложенными конструкциями *od..do*, а двумерные индексы переведены в одномерные по формуле: *индекс в массиве* $= i \cdot N + j$:

```
do
  :: row < N ->
    col = 0;
    do
      :: col < N ->
        sum = 0; k = 0;
        do
          :: k < N ->
            sum = sum + A[row * N + k] * B[k * N + col];
            k = k + 1;
          :: k == N -> break;
        od;
        col = col + 1;
      :: col == N -> break;
    od;
    row = row + P;
  :: row >= N -> break;
od;
```

5. Сборка результатов нулевым процессом

По требованию задания обмен данными (если необходим) ведется каналами, а итоговый результат выводит нулевой процесс. Соответственно, он выступает агрегатором: сам записывает свои строки напрямую в C и принимает результаты остальных воркеров через канал, а затем выводит готовую матрицу.

Объявлен буферизированный канал $N \times N$ сообщений, где одно сообщение – кортеж {номер_строки, номер_столбца, значение}:

```
chan row_computed = [N * N] of { byte, byte, int };
```

Сразу после вычисления ячейки воркеры расходятся на две ветки:

```
if
:: id == 0 -> C[row * N + col] = sum;
:: id != 0 -> row_computed!row,col,sum;
fi;
```

После завершения собственных вычислений 0-й процесс детерминированно определяет, сколько элементов придет по каналу. Из строк $0 \dots N-1$ ему принадлежат те, что удовлетворяют $i \bmod P = 0$ (он считает их сам), остальные вычислены другими воркерами и придут по каналу — по N элементов на строку:

```
expected_elements = 0;
do
:: i < N ->
    if
        :: (i % P) != 0 -> expected_elements =
expected_elements + N;
        :: else -> skip;
    fi;
    i = i + 1;
:: i == N -> break;
od;
```

Далее 0-й процесс в цикле принимает кортежи и раскладывает значения на нужные позиции в C :

```
do
  :: received < expected_elements ->
    row_computed?r,c,val;
    C[r * N + c] = val;
    received = received + 1;
  :: received == expected_elements -> break;
od;
```

Семантика блокировки канала и обеспечивает синхронизацию: операция *row_computed?r,c,val* блокирует процесс, если буфер пуст, и автоматически просыпается, когда отправитель помещает в канал очередной кортеж.

6. Завершение работы и флаг *done*

Программа корректна, если она: (а) завершается и (б) к моменту завершения результирующая матрица C содержит математически верный ответ. Оба эти утверждения должны быть проверяемы формально на языке LTL, для чего требуется глобальный индикатор момента завершения.

Был введен глобальный флаг *done*. Этот флаг устанавливается в *true* только 0-м процессом и только после того, как все строки матрицы C собраны и выведены. Это последняя инструкция блока агрегации.

7. Формирование и верификация LTL-свойств

Для доказательства корректности работы алгоритма и отсутствия тупиковых ситуаций (дедлоков) были сформулированы свойства живости (liveness) и свойство безопасности (safety).

В код введен глобальный флаг `bool done = false`, который переключается в `true` нулевым процессом строго после того, как все вычисления завершены, данные получены и результат выведен.

Свойства сформулированы на языке LTL следующим образом:

```
ltl liveness { <> (done == true) }  
ltl safety { [] (done -> (C[0] == 43 && C[1] == 109 &&  
C[2] == 45 && C[3] == 31 && C[4] == 16 && C[5] == 11 && C[6]  
== 73 && C[7] == 51 && C[8] == 31)) }
```

Формула `liveness` означает: «в любом пути выполнения системы рано или поздно наступит состояние, в котором флаг `done` станет истинным». Формула `safety` означает: «во всех состояниях вычисления: как только флаг `done` становится истинным, значения результирующей матрицы соответствуют ожидаемому результату на тестовых данных».

4. Результаты полной верификации в Spin:

Верификация запускается двумя отдельными командами — по одной на каждое LTL-свойство. Полный обход состояний с проверкой циклов принятия (acceptance cycles) дал следующие результаты.

- Проверка свойства *liveness*:

```
pan: ltl formula liveness
```

Full statespace search for:

never claim	+ (liveness)
assertion violations	+ (if within scope of claim)
acceptance cycles	+ (fairness disabled)
invalid end states	- (disabled by never claim)

State-vector 292 byte, depth reached 455, errors: 0
2301 states, stored (4602 visited)
2923 states, matched
7525 transitions (= visited+matched)
14 atomic steps

unreached in proctype worker
(0 of 93 states)

unreached in init
(0 of 29 states)

unreached in claim liveness
_spin_nvr.tmp:6, state 6, "-end-"
(1 of 6 states)

- Проверка свойства *safety*:
pan: ltl formula safety

Full statespace search for:

never claim	+ (safety)
assertion violations	+ (if within scope of claim)
acceptance cycles	+ (fairness disabled)
invalid end states	- (disabled by never claim)

State-vector 292 byte, depth reached 460, errors: 0
2303 states, stored
313 states, matched
2616 transitions (= visited+matched)
7 atomic steps


```
unreached in proctype worker
    (0 of 93 states)
unreached in init
    (0 of 29 states)
unreached in claim safety
    _spin_nvr.tmp:15, state 10, "-end-"
    (1 of 10 states)
```

Оба свойства верифицированы с результатом *errors: 0*.

Строка *unreached in proctype worker: (0 of 93 states)* подтверждает, что весь код был достижим в ходе проверки — верификатор не пропустил ни одной ветки.

Строки *unreached in claim liveness: (1 of 6 states)* и *unreached in claim safety: (1 of 10 states)* означают, что финальное состояние автомата-ловушки (*never claim*) ни разу не было достигнуто, то есть отрицание обоих свойств невозможно ни в одном из вычислений модели.

5. Сравнение с альтернативным подходом

В ходе работы был проведен анализ решения другой группы, которая реализовывала передачу данных механизмом общей (разделяемой) памяти и отслеживанием статуса выполнения через глобальные переменные.

Ключевые отличия:

- Архитектура синхронизации: В нашем решении синхронизация потоков естественным образом вытекает из блокирующего чтения из канала (`row_computed ? r, c, val`). Агрегатор переходит в режим ожидания (`suspend`) до появления данных в буфере. В решении коллег рабочие процессы осуществляют запись непосредственно в глобальный массив `C`, а синхронизация осуществляется через инкремент глобального атомарного счетчика завершенных задач (`atomic { finished = finished + 1 }`) и последующее "охранное" условие-барьер (`finished == P`), которое блокирует нулевой процесс до момента завершения всех воркеров.

- Производительность: Работа с общей памятью генерирует менее объемное пространство состояний в Spin (меньший размер вектора состояний (State-vector) и меньше transitions), так как симулятору не требуется моделировать и сохранять промежуточные состояния буферов каналов при отправке и чтении каждого отдельного элемента.
- Надежность: Концепция передачи данных через разделяемую память потенциально небезопасна из-за высокого риска состояний гонки (data races). В алгоритме коллег гонок удалось избежать благодаря строгому непересекающемуся разделению по строкам (отсутствует общая перезапись). Однако канальная архитектура (наш вариант) концептуально гораздо надежнее: она снижает связность компонентов и гарантирует потокобезопасность на архитектурном уровне, так как модификацией итоговой структуры C занимается исключительно монопольный процесс-агрегатор (0-й воркер).

Выводы.

В ходе выполнения лабораторной работы был успешно спроектирован, реализован на языке Promela и верифицирован в среде Spin параллельный алгоритм перемножения квадратных матриц. Использование каналов передачи сообщений (chan) показало себя как надежный инструмент синхронизации: нулевой процесс выступал в роли точки синхронизации и успешно агрегировал независимые вычисления рабочих потоков. С помощью строгой проверки LTL-свойств (liveness, safety) было доказано, что созданная математическая модель алгоритма свободна от взаимных блокировок (deadlocks), а полное вычисление матрицы и её вывод гарантированно завершаются при любых возможных вариантах планирования (scheduling) потоков. По результатам обмена опытом с другой группой было установлено, что хотя подход с разделяемой памятью и барьерной синхронизацией генерирует меньшую нагрузку на State-space верификатора, выбранная в нашей работе парадигма передачи сообщений (Message Passing) позволяет писать более масштабируемый и надежный с точки зрения data races распределенный код.

Дополнительно разработанный скрипт автоматического тестирования на Python подтвердил арифметическую корректность возвращаемых алгоритмом данных на матрицах и процессах различной размерности. Полный исходный код программы см. в Приложении А.

ПРИЛОЖЕНИЕ А

Файл main.pml:

```
// Размер квадратных матриц N*N
#define N 3
// Количество параллельных процессов
#define P 2

// Одномерные массивы для хранения двумерных матриц N*N
int A[N * N];
int B[N * N];
int C[N * N];

// Флаг завершения всех вычислений (для верификации LTL)
bool done = false;

// Канал для передачи данных от рабочих процессов к нулевому.
// Передается: { номер строки, номер столбца, вычисленное значение }
chan row_computed = [N * N] of { byte,byte,int };

// Функция рабочего процесса (воркера)
proctype worker(byte id) {
// Процесс берет строки с шагом P, начиная со строки = id
    byte row = id;
    byte col,k;
    int sum;

// Цикл по строкам, закрепленным за данным процессом
    do
        :: row < N ->
            col = 0;
// Цикл по столбцам матрицы B
            do
                :: col < N ->
                    sum = 0;
                    k = 0;
// Вычисление скалярного произведения строки A и столбца B
                    do
                        :: k < N ->
                            sum = sum + A[row * N + k] * B[k * N + col];
                            k = k + 1;
                        :: k == N -> break;
                    od;

// Если это нулевой процесс, он сразу пишет в итоговую матрицу
                    if
                        :: id == 0 ->
                            C[row * N + col] = sum;
// Остальные процессы отправляют результат через канал нулевому процессу
                        :: id != 0 ->
                            row_computed!row,col,sum;
                    fi;
                    col = col + 1;
                :: col == N -> break;
            od;
// Переход к следующей строке, закрепленной за этим процессом
            row = row + P;
        :: row >= N -> break;
    od;

// Только нулевой процесс занимается агрегацией и выводом
    if
```

```

:: id == 0 ->
    int i,j;
    int expected_elements = 0;
    i = 0;
// Подсчитываем, сколько элементов мы должны получить из канала (сколько элементов
вычислили другие процессы)
    do
        :: i < N ->
            if
                :: (i % P) != 0 -> expected_elements = expected_elements + N;
                :: else -> skip;
            fi;
            i = i + 1;
        :: i == N -> break;
    od;

    int received = 0;
    byte r,c;
    int val;
// Цикл ожидания и чтения вычисленных элементов из канала
    do
        :: received < expected_elements ->
            row_computed?r,c,val;
// Запись полученного элемента в результирующую матрицу
        C[r * N + c] = val;
        received = received + 1;
        :: received == expected_elements -> break;
    od;

// Вывод итоговой матрицы C
    printf("Result Matrix C:\n");
    i = 0;
    do
        :: i < N ->
            j = 0;
            printf("[");
            do
                :: j < N ->
                    if
                        :: j < N - 1 -> printf("%d,",C[i * N + j]);
                        :: j == N - 1 -> printf("%d",C[i * N + j]);
                    fi;
                    j = j + 1;
                :: j == N -> break;
            od;
            printf("]\n");
            i = i + 1;
        :: i == N -> break;
    od;

// Устанавливаем флаг завершения работы
    done = true;
    :: id != 0 -> skip;
    fi;
}

init {
// Тестовые данные
    A[0] = 5;A[1] = 3;A[2] = 7;
    A[3] = 1;A[4] = 4;A[5] = 0;
    A[6] = 3;A[7] = 9;A[8] = 1;

```

```

        V[0] = 3;V[1] = 8;V[2] = 7;
        V[3] = 7;V[4] = 2;V[5] = 1;
        V[6] = 1;V[7] = 9;V[8] = 1;
// Ожидаемый результат
// [43,109,45]
// [31,16,11]
// [73,51,31]

// Атомарный запуск P рабочих процессов
int proc_id = 0;
atomic {
    do
        :: proc_id < P ->
            run worker(proc_id);
            proc_id = proc_id + 1;
        :: proc_id == P -> break;
    od;
}

// Свойство живости
ltl liveness { <> (done == true) }

// Свойство безопасности
ltl safety { [] (done -> (C[0] == 43 && C[1] == 109 && C[2] == 45 &&
    C[3] == 31 && C[4] == 16 && C[5] == 11 &&
    C[6] == 73 && C[7] == 51 && C[8] == 31)) }

```